

Министерство науки и высшего образования Российской Федерации ФГБОУ
ВО «Российский экономический университет имени Г.В. Плеханова»
Пермский филиал

ПРАКТИЧЕСКАЯ РАБОТА №2

по дисциплине

«Поддержка и тестирование программных модулей»

Тема: Разработка приложения «Калькулятор»

Выполнил: студент группы ИПс-21

Исмаилов Даниил

Проверил: Берестов Д.Б.

Пермь, 2026

1. Введение

Настоящая практическая работа посвящена разработке настольного приложения «Калькулятор» на платформе .NET 8 с использованием технологии WPF (Windows Presentation Foundation). Работа выполнена в соответствии с заданием по дисциплине «Поддержка и тестирование программных модулей».

Цель работы — создать приложение-калькулятор, реализующее вычисление арифметических выражений с поддержкой операторов +, -, *, /, ^ (возведение в степень) и скобок. Основная логика вынесена в отдельную библиотеку классов; интерфейс реализован в виде WPF-приложения по паттерну MVVM. Для библиотеки написаны модульные тесты.

2. Используемые технологии

В ходе разработки использовались следующие технологии и инструменты:

Компонент	Технология
Платформа	.NET 8
UI-фреймворк	WPF (net8.0-windows)
Библиотека логики	Class Library (net8.0)
Тестирование	MSTest
Сериализация	System.Text.Json
IDE	Visual Studio 2022
Система контроля версий	Git / GitHub

3. Структура проекта

Решение состоит из трёх проектов:

- IsmailovCalculatorLib — библиотека классов с бизнес-логикой.
- IsmailovCalculator — WPF-приложение (пользовательский интерфейс).
- IsmailovCalculatorTests — проект модульных тестов (MSTest).

Такое разделение соответствует принципу разделения ответственности: библиотека не зависит от WPF и может быть протестирована изолированно; приложение зависит только от библиотеки и не содержит бизнес-логики.

Внутренняя структура IsmailovCalculatorLib:

- Engine/ — алгоритм вычисления выражений (ExpressionEvaluator.cs).
- Models/ — модели данных (CalculationHistoryItem.cs, ThemeSettings.cs).
- Services/ — сервисы персистентности (HistoryService.cs, SettingsService.cs).
- Infrastructure/ — вспомогательные классы (AppPaths.cs)

4. Создание проекта в Visual Studio

Для создания решения в Visual Studio 2022 был выполнен следующий порядок

действий:

1. Создан новый Solution с именем IsmailovCalculator.
2. Добавлен проект Class Library (.NET 8) с именем IsmailovCalculatorLib.
3. Добавлен проект WPF Application (.NET 8) с именем IsmailovCalculator.
4. Добавлен проект MSTest Test Project с именем IsmailovCalculatorTests.
5. Добавлены ссылки на проекты: IsmailovCalculator → IsmailovCalculatorLib;
IsmailovCalculatorTests → IsmailovCalculatorLib.

5. Реализация библиотеки классов

5.1 Алгоритм вычисления выражений

Для разбора и вычисления арифметических выражений реализован алгоритм Shunting-yard (алгоритм сортировочной станции). Алгоритм переводит инфиксную запись выражения в обратную польскую запись (RPN), после чего вычисляет результат через стек операндов.

Реализация разбита на три этапа:

1. **Tokenize** — токенизация строки: выделение чисел, операторов и скобок.
2. **ToRPN** — перевод списка токенов в обратную польскую запись.
3. **EvaluateRPN** — вычисление RPN-выражения с использованием стека.

Публичный метод `ExpressionEvaluator.Evaluate` объединяет все три этапа:

```
public static decimal Evaluate(string expression)
{
    if (string.IsNullOrEmpty(expression))
        throw new FormatException("Выражение не может быть пустым.");

    // Нормализация унарного минуса и знаков
    if (expression.StartsWith("-") || expression.StartsWith("+"))
        expression = "0" + expression;
    expression = expression.Replace("-", "(0-").Replace("+", "(0+");

    var tokens = Tokenize(expression);
    var rpn = ToRPN(tokens);
    return EvaluateRPN(rpn);
}
```

Для хранения чисел используется тип `decimal` вместо `double`, что исключает ошибки представления чисел с плавающей точкой. `OverflowException`.

5.2 Модели и инфраструктура

Для управления файлами конфигурации и истории используется класс `AppPaths`:

```
internal static class AppPaths
{
    private const string AppFolderName = "IsmailovCalculator";
    public static string GetFilePath(string fileName)
    {
        var baseDir =
            Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData);
        return Path.Combine(baseDir, AppFolderName, fileName);
    }
}
```

Файлы истории (`history.json`) и настроек (`settings.json`) сохраняются в пути `%LocalAppData%\IsmailovCalculator\`.

6.1 Архитектура MVVM

Приложение построено по паттерну MVVM (Model-View-ViewModel):

- **Model** — классы библиотеки `IsmailovCalculatorLib`.
- **ViewModel** — `MainViewModel`, реализует `INotifyPropertyChanged`.
- **View** — `MainWindow.xaml` с привязками данных и кастомным XAML-стилем.

6.2 Интерфейс и графический стиль

Графический интерфейс выполнен в уникальном кастомном стиле с темной цветовой палитрой. Интерфейс содержит:

- Дисплей с предпросмотром текущего выражения и крупным шрифтом вывода результата.
- Кастомные стили кнопок с анимацией наведения (`IsMouseOver`) и нажатия (`IsPressed`).
- Цветовое разделение функций: бирюзовые акценты для операторов, розовые — для кнопок очистки, фиолетовый — для кнопки выполнения вычисления (`=`).

7. Модульное тестирование

Для библиотеки `IsmailovCalculatorLib` написаны модульные тесты с использованием фреймворка **MSTest**. Тесты покрывают основные математические и граничные сценарии.

7.1 Структура тестов

Тесты размещены в классе `EvaluatorTests` с использованием атрибутов `[TestClass]`,

[TestMethod] и [DataRow]:

.

7.1 Структура тестов

Тесты организованы в группы с использованием атрибутов [Fact] для одиночных тестов и [Theory] + [InlineData] для параметризованных:

Группа тестов	Что проверяется
Базовые операции	Сложение, вычитание, умножение, деление и степень
Унарный минус	Отрицательные числа в начале строки и внутри скобок
Приоритет и скобки	Соблюдение порядка математических операций
Исключения	Деление на ноль, пустая строка, несбалансированные скобки

7.2 Результаты тестирования

Все 11 тест-кейсов успешно прошли проверку в Test Explorer Visual Studio 2022.

Запуск выполняется командой:

```
dotnet test IsmailovCalculator.sln -c Release
```

Итого: 11 пройденных тестов, 0 ошибок, 0 пропущенные.

8. Размещение на GitHub

Исходный код проекта размещён в публичном репозитории на GitHub.

Репозиторий содержит:

1. Исходный код всех трёх проектов solution.
2. Оформленный файл README.md с описанием архитектуры и инструкциями по запуску.
3. Файл лицензии LICENSE (MIT).

Для публикации использовались команды:

```
git init
git add .
git commit -m "Initial commit IsmailovCalculator"
git branch -M main
git remote add origin https://github.com/<username>/IsmailovCalculator.git
git push -u origin main
```

9. Заключение

В ходе практической работы было разработано настольное приложение «Калькулятор» на платформе .NET 8 с использованием технологии WPF. Все требования задания выполнены:

- Реализована поддержка арифметических операторов +, -, *, /, ^ и скобок.
- Логика вычислений вынесена в отдельную библиотеку IsmailovCalculatorLib.
- Интерфейс выполнен в соответствии с паттерном MVVM.
- Написан комплекс модульных тестов на MSTest.
- Название проектов содержит фамилию студента (IsmailovCalculator).
- Код и документация подготовлены для размещения на GitHub.

